

Software Engineering Course: Term Project Assignment Sequence

Project Title: *Conceptualization and Process Design for a [Your Choice of System]*

Objective: The goal of this term project is to synthesize the principles of software engineering covered in class. You will select a software system idea and develop a comprehensive project dossier that walks through its initial conceptualization, requirement analysis, process model selection, design, testing strategy, and consideration of evolution and ethics. This is primarily a design and planning exercise, not a coding project.

Deliverable Structure: Students will produce a single, well-structured report document divided into phases, submitted progressively or as a final draft.

Phase 1: Project Initiation & Abstract

Objective: To define the project's core idea at a high level and establish its purpose.

Task:

1. **Select a System:** Choose a software system you would like to conceptualize. Examples:
 - A smart campus navigation app for new students.
 - An online platform for managing local community service projects.
 - A personal finance tracker with budgeting goals.
 - A system for a small library to manage book loans and inventory.
2. **Write an Abstract:** Develop a one-paragraph abstract (approx. 150-200 words) that summarizes the entire project. It must briefly state:
 - The **problem** the software aims to solve.
 - The **primary objectives** and key features of the proposed system.
 - The **intended users** (who will benefit from it).
 - The **significance** of the project (why it is useful or innovative).

Learning Outcome: Students learn to concisely communicate the essence of a software project.

Phase 2: Introduction & Feasibility Study

Objective: To provide a detailed background and justify the project's viability.

Task:

1. **Write a Detailed Introduction:** Expand on the abstract. This section should include:
 - The problem statement in detail.
 - The proposed solution (your system).
 - The project's scope and key objectives.
 - A brief description of the key stakeholders (users, clients, etc.).
2. **Conduct a Mini Feasibility Study:** Write a short analysis covering:
 - **Technical Feasibility:** What technologies (platforms, languages, databases) might be used? Are they readily available?
 - **Economic Feasibility:** What are the potential benefits vs. costs? (Think at a high level).
 - **Operational Feasibility:** Will the intended users be able and willing to use this system?

Learning Outcome: Students practice contextualizing a project and critically analyzing its practicality before deep investment.

Phase 3: Software Specification (Requirements Engineering)

Objective: To formally define *what* the system must do.

Task:

1. **Elicit and List Requirements:** Create a list of functional and non-functional requirements.
 - **Functional Requirements:** (e.g., "The user shall be able to search for a book by title, author, or ISBN.")
 - **Non-Functional Requirements:** Address attributes from the lectures: **Usability, Reliability, Performance, Security.**
2. **Create Use Case Diagrams (Optional but Recommended):** Model the interactions between actors (users) and the system using a use case diagram for the key functionalities.
3. **Write a User Story:** Write one detailed user story for a core feature, following the format: "As a [type of user], I want to [some goal], so that [some reason]."

Learning Outcome: Students gain hands-on experience with the critical process of requirements engineering.

Phase 4: Software Process Model Selection & Justification

Objective: To select an appropriate development methodology and justify the choice.

Task:

1. **Select a Process Model:** Choose one of the primary process models covered in the lectures:
 - Waterfall Model
 - Incremental Development
 - Reuse-Oriented Software Engineering (Component-Based)
2. **Justify Your Choice:** Write a detailed justification explaining *why* your chosen model is the most suitable for your specific project. Consider:
 - Stability of the requirements you defined in Phase 3.
 - Project size and complexity.
 - Need for early user feedback.
 - Potential for using existing components or services.

Learning Outcome: Students understand that there is no "one-size-fits-all" process and learn to make informed methodological decisions.

Phase 5: System Design

Objective: To create a high-level blueprint of the system architecture.

Task:

1. **Architectural Design:** Describe the high-level architecture of your system (e.g., Client-Server, 3-Tier, Model-View-Controller). Use a simple block diagram to illustrate the main components.
2. **Component Design:** Identify and describe 3-4 key software components (e.g., "User Authentication Module," "Data Search Engine," "Reporting Component"). Briefly state the responsibility of each.
3. **UI/UX Wireframe (Optional but Recommended):** Create a simple hand-drawn or digital wireframe for one key screen of your application, explaining the user interaction flow.

Learning Outcome: Students practice translating requirements into a structural design.

Phase 6: Validation & Testing Strategy

Objective: To plan how the software will be tested to ensure quality.

Task:

1. **Define a Testing Strategy:** Outline your approach to the different testing stages:
 - **Unit Testing:** What will be tested at the component level? (e.g., "The function that calculates late fees will be unit tested.")
 - **Integration Testing:** How will you test the interaction between your key components?
 - **System Testing:** How will you validate the non-functional requirements (e.g., performance under load, usability)?
 - **Acceptance Testing:** Describe a scenario for UAT. What would a user do to sign off that the system meets their needs?

Learning Outcome: Students learn that testing is a structured, planned activity integral to the software process.

Phase 7: Evolution, Professional & Ethical Responsibility

Objective: To consider the long-term lifecycle of the software and the ethical implications of its development.

Task:

1. **Software Evolution Plan:** Propose two potential future changes or enhancements that could be made to the system after its initial release. Explain how your chosen process model from Phase 4 would accommodate these changes.
2. **Ethical Analysis:** Analyze your project against **two principles** from the ACM/IEEE Code of Ethics (e.g., *Public Interest, Client and Employer, Product*). For example, if your app collects user data, how do you uphold the "Public Interest" principle regarding privacy and security?

Learning Outcome: Students develop a long-term perspective on software and recognize their professional ethical responsibilities.

Final Submission

The final report should be a consolidated document containing all seven phases, presenting a complete and well-reasoned plan for the proposed software system.

This sequence ensures that students engage deeply with every major topic from your lectures, from the initial "What is software engineering?" to the final "How do we maintain and improve software responsibly?".